

Design and Implementation of 32 Bit and 64 Bit Vedic Multiplier

K Kushanth Varma

Dept. of ECE

Chaitanya Bharathi Institute of
Technology
Hyderabad, Indiakushanthvarma.k@gmail.com

M Lakshmi Narasimha Charyulu

Dept. of ECE

Chaitanya Bharathi Institute of
Technology
Hyderabad, Indiamlncharyulu_ece@cbit.ac.in

Vivek Singh Kushwah

Dept. of ECE

Chaitanya Bharathi Institute of
Technology
Hyderabad, Indiadrviveksinghkushwah@gmail.com

Abstract--- The proposed research introduces a modified 64-bit binary Vedic multiplier, derived from the previously implemented 32-bit binary Vedic multiplier based on ancient Indian Vedic mathematics. This enhanced 64-bit design improves upon the earlier 32-bit architecture [1] to achieve higher performance. The modified Vedic multiplier demonstrates significant improvements, particularly in terms of reduced propagation delay and optimized device utilization. The architecture was designed using Verilog HDL, and Vivado was employed for both simulation and synthesis. Multiplication operations were simulated for 4-bit, 8-bit, 16-bit, 32-bit, and 64-bit configurations. The previously developed 32-bit Vedic multiplier [1] featured a scalable design, which enabled its extension to 64-bit operations in this work. Comparative analysis with conventional Vedic multipliers reveals that the proposed modified design achieves notable enhancements in delay performance.

Keywords--- full adder, ripple carry adder, carry save adder, CMOS.

I. INTRODUCTION

Multiplication is a fundamental operation in digital electronics and plays a vital role in applications such as calculators, mobile devices, computers, image processing, and digital signal processing. In this work, a 64×64 Vedic multiplier is designed using the Urdhva Tiryagbhyam sutra from Vedic mathematics, which translates to “vertical and crosswise.” The design is implemented in Verilog HDL and verified using Xilinx Vivado tools. The large 64×64 multiplication is hierarchically decomposed into smaller blocks of 32×32 Vedic multipliers, improving both speed and efficiency. This modular approach helps reduce computational delay and hardware utilization, making it suitable for high-speed digital systems. The main objectives of this project are to design, implement, and analyze 32-bit and 64-bit Vedic multipliers using a hierarchical, parameterized Verilog model, and to compare their performance in terms of delay and utilization with previously published 8-bit, 16-bit, and 32-bit designs. Additionally, the work includes an understanding of the 4-bit Ripple Carry Adder (RCA) as a fundamental arithmetic building block for the proposed architecture.

II. LITERATURE

Numerous studies have aimed to improve the performance of digital multipliers by incorporating concepts from Vedic mathematics and other advanced computational techniques to overcome the limitations of conventional array-based multipliers. Among these approaches, the Urdhva Tiryagbhyam sutra is one of the most widely utilized due to its ability to generate partial products in parallel and its inherently modular structure, which supports scalability for higher bit-width operations. Bala Muruges et al [1] proposed an enhanced 32-bit Vedic multiplier by replacing Ripple Carry Adders (RCA) with Carry Save Adders (CSA) to reduce intermediate computation delay. Their design, implemented in Verilog HDL and synthesized using Xilinx ISE, achieved a delay of 68.859 ns, compared to 80.783 ns in the conventional architecture.

However, the speed improvement came at the cost of slightly higher logic utilization, illustrating the typical trade-off between area and performance in VLSI systems. In a related comparative study [2], Vedic multipliers were evaluated against Booth and array multipliers. Although Booth’s algorithm efficiently handles signed multiplications and minimizes partial products, Vedic multipliers demonstrated superior speed and better modular scalability. Collectively, existing research indicates that Booth multipliers are more efficient in terms of power and resource usage for large operands, whereas Vedic multipliers offer predictable scalability with consistently lower delay. Building upon these findings, the present work implements a hierarchical 64×64 Vedic multiplier using Verilog HDL. The architecture is constructed from smaller reusable modules—4×4, 8×8, 16×16, and 32×32—to ensure modularity and efficient resource utilization. Additionally, the incorporation of CSAs in higher levels of the hierarchy minimizes propagation delay, allowing the proposed design to outperform previous implementations in terms of speed while maintaining scalability and hardware efficiency.

III. DESIGN

A. Ripple Carry Adder

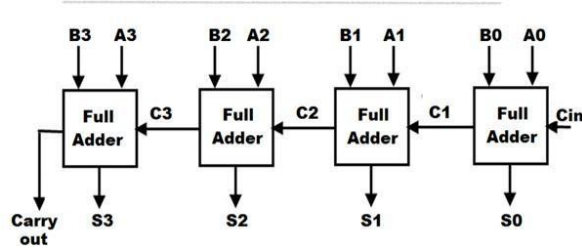


Fig 1. Block diagram of 4-bit Ripple Carry Adder

Ripple carry adder is a digital system made up of full adder blocks [6]. For this paper we are studying about 4-bit ripple carry adder so this includes 4 full adder blocks as shown in the above figure 1. C_{in} is initial carry to first full adder which usually is zero. S_1, S_2, S_3, S_4 are the sum of the respective full adder blocks similarly C_1, C_2, C_3, C_{out} are its carry. From the figure we can say that the output of first full adder is input to second full adder and so on this is like the outputs are getting rippled from one full adder to the other hence it is called ripple carry adder. The design has multiple blocks and each block has some delay in processing. Assuming each block introduces a delay of 10 ns, the total delay for the 4-bit RCA would be approximately 40 ns.

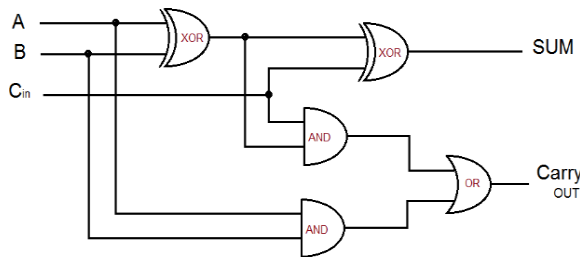


Fig 2. Full Adder logic design

The ripple carry adder has a full adder as shown in figure 2 [7], which is a combinational logic block in digital design which has 3 inputs A, B, C_{in} and 2 outputs sum, C_{out} . This has 8 combinations of output and it does not have any memory of previous output only works on current input and gives output so its combinational circuit and to calculate the Sum & C_{out} of the full adder we have these expressions in reference to table

Table 1. Truth Table of 1-bit Full Adder

Inputs			Outputs	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0

0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

From the truth table, the logical expression for the sum in a full adder is:

$$Sum = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$$

Since $A'B + AB' = A \oplus B$. This simplifies to: $Sum = C_{in}(A \oplus B)' + C_{in}'(A \oplus B)$

The final simplified expression is:

$$Sum = A \oplus B \oplus C_{in}$$

From the truth table, the logical expression for C_{out} in a full adder is:

$$C_{out} = A'B C_{in} + A B' C_{in} + A B C_{in}' + A B C_{in}$$

This simplifies to:

$$C_{out} = A B (C_{in}' + C_{in}) + C_{in}(A'B + AB')$$

Since $C_{in}' + C_{in} = 1$ and $A'B + AB' = A \oplus B$. Thus, the final simplified expression is:

$$C_{out} = A B + C_{in} (A \oplus B)$$

The full adder performs addition using current inputs only, with outputs defined by $Sum = A \oplus B \oplus C_{in}$ and $C_{out} = AB + C_{in} (A \oplus B)$. It is a key element in ripple carry adders and forms the basis for complex arithmetic circuits.

B. Carry Save Adder

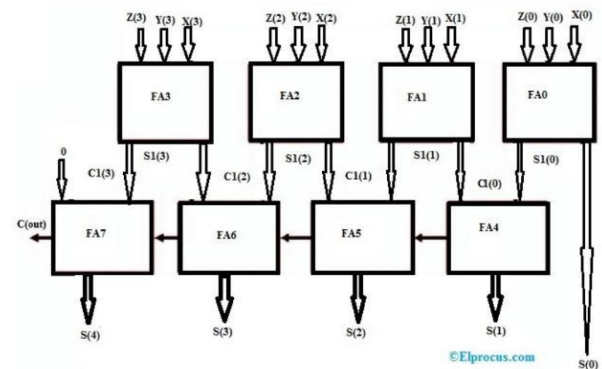


Fig 3. Carry Save Adder

A **Carry Save Adder (CSA)** is a specialized digital adder designed to efficiently handle the addition of three or more binary numbers. Unlike conventional adders such as Ripple Carry Adders (RCA) [8], where the carry must propagate through every stage before producing the final result, a CSA

reduces delay by storing carry bits instead of immediately propagating them. This makes it particularly useful in applications like high-speed multiplication and accumulation, where multiple partial products need to be summed quickly..

Working Principle

In a CSA, three binary operands (often labeled X, Y, and Z) are fed into an array of Full Adders (FAs). For example, when working with 4-bit numbers, four full adders are used—one for each bit position. Each full adder computes a **sum** and a **carry** output:

- **Sum:** $S_i = X_i \oplus Y_i \oplus Z_i$
- **Carry:** This is saved, not propagated immediately

These carry bits are stored and used in a subsequent stage rather than being sent directly to adjacent adders. In this next stage, typically implemented with a ripple carry adder, the stored carry and sum bits are finally added to produce the complete result. For example, if two 32-bit binary numbers (X and Y) are processed using a CSA, 32 full adders are arranged in parallel. Each full adder independently computes a partial sum (S) and carry (C) based on the corresponding bits of the inputs:

- $S_i = X_i \oplus Y_i$
- $C_i = X_i \cdot Y_i$

C. Block Diagram of Hierarchical Vedic Multiplier

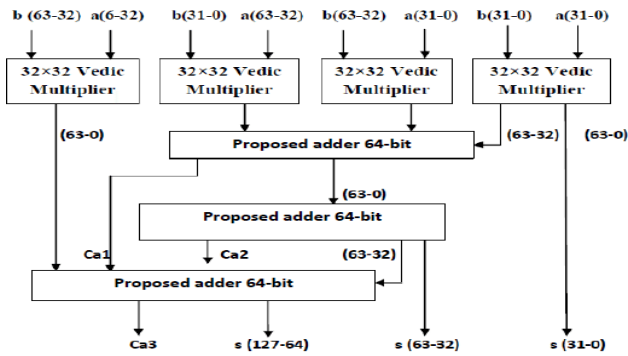


Fig 4. Hierarchical Vedic Multiplier

The 64×64 multiplier is realized using four 32×32 Vedic multipliers. The 64-bit inputs are split into high and low halves, and partial products are generated in parallel. These outputs are properly shifted and combined through a sequence of 64-bit adders to produce the final 128-bit result [9]. This modular structure improves speed, supports scalability, and reduces delay by reusing smaller multiplier blocks. The 64×64 multiplier is realized using four 32×32 Vedic multipliers. The 64-bit inputs are split into high and

low halves, and partial products are generated in parallel. These outputs are properly shifted and combined through a sequence of 64-bit adders to produce the final 128-bit result [10]. This modular structure improves speed, supports scalability, and reduces delay by reusing smaller multiplier blocks

IV. RESULT & DISCUSSION

Simulated results are being plotted in the graphs and timing diagram. In figure 5 delay graph is shown in figure 6 LUT comparative graph and timing diagram of 64x64 Vedic Multiplier in figure 7. The table 2 shows the comparative analysis.

Table 2. Delay and LUT comparison

Bit-width	Model Type	Delay (ns)	LUTs
4×4	Published	14.215	38
	Proposed	9	16
8×8	Published	24.186	201
	Proposed	13	80
16×16	Published	43.714	903
	Proposed	17	348
32×32	Published	68.859	3802
	Proposed	25	1424
64×64	Published	Not Available	Not Available
	Proposed	33	4356

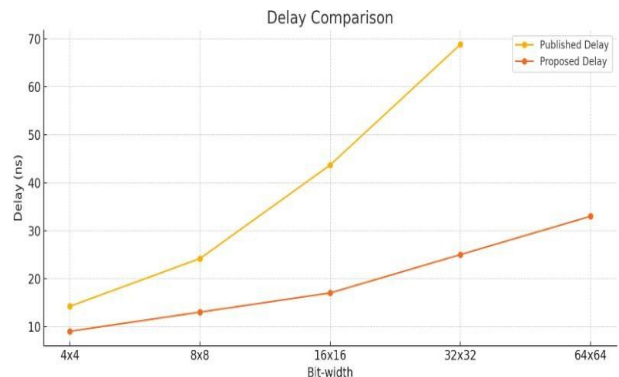


Fig 5. Delay Comparison Graph

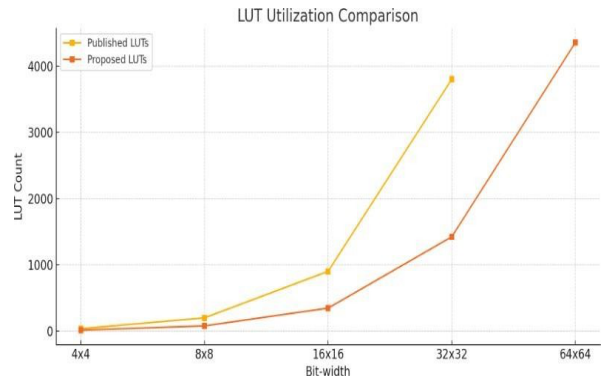


Fig 6. LUT Comparison Graph

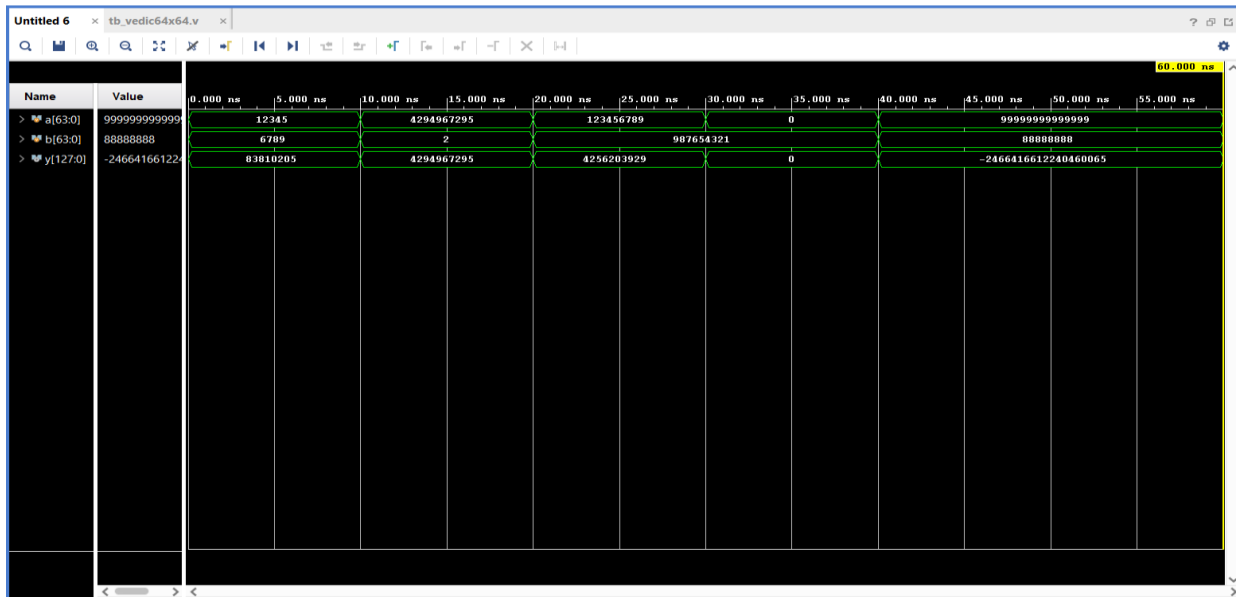


Fig 6. Simulation of 64x64 Vedic Multiplier

Table 3 Different Multiplier Comparison Table

Multiplier Type	Delay (ns)	Area (LUTs)	Power (W)
Vedic (Proposed)	19.14	4356	1.115
Wallace Tree	25–30	~5000	1.2
Booth Multiplier	28–35	~4800	1.25
Array Multiplier	35–40	~5200	1.3

Table 3 is showing different multipliers' comparison on the basis of their outcome.

V. CONCLUSION

In this work we successfully designed and implemented Vedic multipliers up to 64×64 bits using the Urdhva Tiryagbhyam sutra with a hierarchical approach, starting from 4×4 and extending through 8×8 , 16×16 , and 32×32 modules [11]. Each stage was verified with Verilog HDL testbenches to ensure accuracy and reliability. The results demonstrated better speed and modularity compared to conventional multipliers, while also providing practical experience in structural modeling, coding, and verification. The 64×64 multiplier, currently under final validation, is expected to preserve the performance gains observed in the smaller modules reinforcing the potential of Vedic mathematics in modern digital design

REFERENCE

- [1] M. B. Muruges, S. Nagaraj, J. Jayasree, and G. Vijay Kumar Reddy, "Modified High Speed 32-bit Vedic Multiplier Design and Implementation," in Proc. Int. Conf. on Electronics and Sustainable Communication Systems (ICESC 2020), IEEE, 2020, pp. 929932, doi:10.1109/ICESC48915.2020.9155924.
- [2] P. Singh, R. Bansal, N. Sachdeva, and P. Dimri, "High Speed 64 Bit Vedic & Booth Multiplier Implementation Using FPGA," in Proc. 3rd IEEE Int. Conf. on Artificial Intelligence for Internet of Things (AIIoT 2024), IEEE, 2024, pp. 1–6, doi: 10.1109/AIIoT58432.2024.10574654
- [3] S. Akhter, "VHDL implementation of fast $N \times N$ multiplier based on Vedic mathematics," in Proc. 18th European Conference on Circuit Theory and Design, 2007, pp. 472–475.
- [4] S. Nagaraj, Dr G.M. Sreerama Reddy and Dr S. Aruna Mastani A Comparative Study on Different Multipliers-Survey Journal of Advanced Research in Dynamical and Control Systems 14739–7522018 Institute of Advanced Scientific Research.
- [5] M. Pushpa, S. Nagaraj, Design and Analysis of 8-bit Array, Carry Save Array, Braun, Wallace Tree and Vedic Multipliers, IEEE Sponsored International Conference On New Trends In Engineering & Technology (ICNTET 2018)
- [6] Nagaraj, S Thyagarajan, K ; Srihari, D ; Gopi K, Design and Analysis of Wallace Tree Multiplier for CMOS and CPL Logic2018 International Conference on Computation of Power, Energy, Information and Communication (ICCPIC) 006–0102018
- [7] S. Nagaraj, Dr.G.M. Sreerama Reddy and Dr.S. Aruna Mastani; A Survey on Adiabatic Logic International Conference on Communications, Signal Processing and VLSI(IC2SV2019), Springer Conference , National Institute of Technology, Warangal.

- [8.] S. Nagaraj, K. Venkataramana Reddy and P. Anil Kumar; Analysis of Vedic Multiplier for Conventional CMOS & Complementary Pass Transistor Logic(CPL) Logics SCOPUS Indexed Springer 8th International Conference on Innovations in Electronics and Communication Engineering, (ICIECE-2019)
- [9.] Au L.S. and Burgess N. (2002), "A (4:2) adder for unified GF(p) and GF(2n) Galois field Multipliers", Proceedings of 36th IEEE Asilomar Conference on Signals, Systems, and Computers, vol. 2, pp. 1619-1623. pp. 299 – 304.
- [10.] Josmin Thomas ; R. Pushpangadan ; S Jinesh Comparative study of performance Vedic multiplier on the Basis of Adders used 2015 IEEE International WIE Conference on Electrical and Computer Engineering (WIECON-ECE
- [11.] GS Tomar, Marcus L George, "Modified Binary Multiplier Architecture to Achieve Reduced Latency and Hardware Utilization", Wireless Personal Communication, Vol.98, No.4, pp.3549-3561, 2018.
- [12.] K. N. Singh and H. Tarunkumar, "A review on various multipliers designs in VLSI," 2015 Annual IEEE India Conference (INDICON), New Delhi, India, 2015, pp. 1-4, doi: 10.1109/INDICON.2015.7443420.